

AG35-CET QuecOpen(SDK) eMMC 开发指导

LTE Standard 模块系列

版本：1.0.0

日期：2023-11-20

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登陆网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他软硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2023，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2023.

文档历史

修订记录

版本	日期	作者	变更表述
-	2023-11-20	Gabriel LI	文档创建
1.0.0	2023-11-20	Gabriel LI	临时版本

目录

文档历史	3
目录	4
表格索引	5
图片索引	6
1 引言	7
2 eMMC 硬件接口及引脚定义	8
3 eMMC 设备树配置及内核配置	9
3.1. 设备树文件配置	9
3.2. 内核配置	10
3.2.1. eMMC 驱动配置	10
3.2.2. ext4 文件系统	10
4 eMMC 文件系统制作和功能测试验证	12
4.1. LTE OPEN EVB 硬件测试环境	12
4.2. eMMC 设备识别	12
4.3. eMMC 设备块大小	12
4.4. 分区管理	13
4.4.1. 查看 eMMC 已有分区	13
4.4.2. 删除分区	13
4.4.3. 创建分区	14
4.4.4. 写入新分区信息	14
4.4.5. 格式化分区为 ext4 文件系统	15
4.5. 分区读写测试	15
5 eMMC 常用调试方法	17
5.1. eMMC 设备读写测试	17
5.1.1. 写入数据	17
5.1.2. 读取数据	17
5.2. 查看当前系统支持的文件系统	18
5.3. 获取 eMMC 基本信息	18
5.4. Linux MMC 驱动子系统简述	19
5.4.1. Linux MMC 子系统介绍	19
5.4.2. Linux MMC 子系统驱动初始化流程	20
5.5. 打开调试信息	21
5.5.1. 打开 MMC 子系统调试信息	21
5.5.2. 打印 MMC 命令行 CMD log 信息	21
6 Mdev 自动挂载	22
6.1. mdev 简介	22
6.2. eMMC 自动挂载机制	22
7 附录 参考文档及术语缩写	24

表格索引

表 1: SDIO 接口引脚定义.....	8
表 2: 参考文档.....	24
表 3: 术语缩写.....	24

Preliminary
Quectel Confidential

图片索引

图 1: Linux MMC 子系统架构图	19
图 2: Linux MMC 驱动初始化流程图	20

Preliminary
Quectel Confidential

1 引言

移远通信 AG35-CET 模块支持 QuecOpen®方案。QuecOpen®是基于 Linux 的嵌入式开发平台，可简化 IoV 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]。

SD 卡即安全数据存储卡，广泛应用于便携式设备上，例如数码相机、个人数码助理和多媒体播放器等。

eMMC 为嵌入式多媒体存储卡。MMC 表示多媒体存储卡，是一种闪存卡（Flash Memory Card）标准，它定义了 MMC 的架构以及访问 Flash Memory 的接口和协议。而 eMMC 则是对 MMC 的一个拓展，以满足更高标准的性能、成本、体积、稳定、易用等需求。

Linux Kernel 使用 MMC 子系统统一管理 eMMC、SD 卡、SDIO Wi-Fi 设备。eMMC 强调的是多媒体存储；SD 强调的是安全和数据保护；SDIO 是从 SD 演化出来的安全数字输入输出接口。为了满足用户多元化的需求，移远通信 AG35-CET QuecOpen 模块预留一路 SDIO 接口用于连接 eMMC。该 SDIO 接口符合 SD 3.0 协议。

本文档适用于 SDK 构建环境的 QuecOpen®方案，主要介绍在 QuecOpen®方案下，移远通信 AG35-CET 模块 eMMC 开发流程，包括接口引脚定义、设备树设置、分区管理以及调试方法，以帮助用户简易且快速地进行开发。

2 eMMC 硬件接口及引脚定义

AG35-CET 模块提供了一路支持 SD 3.0 协议的 SDIO 接口，可用于外接 eMMC 设备。具体引脚定义如下表所示：

表 1：引脚定义

引脚名	引脚号	I/O	描述
SDIO2_VDD	46	PO	SDIO 电源
SDIO2_DATA0	49	DIO	SDIO 数据位 0
SDIO2_DATA1	50	DIO	SDIO 数据位 1
SDIO2_DATA2	47	DIO	SDIO 数据位 2
SDIO2_DATA3	48	DIO	SDIO 数据位 3
SDIO2_CMD	51	DIO	SDIO 命令
SDIO2_CLK	53	DO	SDIO 时钟
EMMC_RST_N	52	DI	eMMC 复位

有关详细信息，请参考文档 [2]。

3 eMMC 设备树配置及内核配置

3.1. 设备树文件配置

参考如下代码，修改设备树文件 `ql-ol-kernel/arch/arm/boot/dts/ql-1806e-std.dts` 的设备树节点 `sdh0` 的配置信息。

```
/* For 1 to support emmc(default), for 0 to support SD card */
#define CONFIG_USE_EMMC      1
#define CONFIG_USE_EMMC_SD_ADAPTIVE    0

#if (CONFIG_USE_EMMC)
&sdh0 {
    pinctrl-0 = <&sdh0_pmx_func1 &sdh0_pmx_func2>;
    pinctrl-1 = <&sdh0_pmx_func1_slow &sdh0_pmx_func2_slow>;
    pinctrl-2 = <&sdh0_pmx_func1_fast &sdh0_pmx_func2_fast>;
    /delete-property/ no-mmc;
    no-sd;
    asr,sdh-host-caps2 = <(
        MMC_CAP2_ONLY_1_8V
    )>;

/* prop "sdh-dtr-data":<timing preset_rate src_rate tx_delay rx_delay tx_dline_reg rx_dline_reg> */
    asr,sdh-dtr-data =
        <PXA_MMC_TIMING_LEGACY PXA_SDH_DTR_26M PXA_SDH_DTR_104M 0 0 0 0>,
        <PXA_MMC_TIMING_MMC_HS PXA_SDH_DTR_52M PXA_SDH_DTR_104M 0 0 0 0>,
        <PXA_MMC_TIMING_UHS_DDR50 PXA_SDH_DTR_52M PXA_SDH_DTR_104M 0 0 0 3>,
        <PXA_MMC_TIMING_MMC_HS200 PXA_SDH_DTR_208M PXA_SDH_DTR_208M 0 0 0 0>,
        <PXA_MMC_TIMING_MAX PXA_SDH_DTR_PS_NONE PXA_SDH_DTR_104M 0 0 0 0>;

/* Enable the emmc and SD adaptive functions */
    #if (CONFIG_USE_EMMC_SD_ADAPTIVE)
    /delete-property/ no-sd;
    quec-emmc-sd-gpio = <&gpio 43 0>;
    /*GPIO43 is used as the eth power enable pin by default. Customers need to select a proper pin*/
    asr,sdh-dtr-data-sd=
```

```
<PXA_MMC_TIMING_LEGACY PXA_SDH_DTR_26M PXA_SDH_DTR_104M 0 0 0 0>,
<PXA_MMC_TIMING_SD_HS PXA_SDH_DTR_52M PXA_SDH_DTR_104M 0 0 0 0>,
<PXA_MMC_TIMING_UHS_DDR50 PXA_SDH_DTR_52M PXA_SDH_DTR_104M 0 0 0 3>,
<PXA_MMC_TIMING_UHS_SDR50 PXA_SDH_DTR_104M PXA_SDH_DTR_208M 0 0 0 0>,
<PXA_MMC_TIMING_UHS_SDR104 PXA_SDH_DTR_208M PXA_SDH_DTR_208M 0 0 0
0>,
<PXA_MMC_TIMING_MAX PXA_SDH_DTR_PS_NONE PXA_SDH_DTR_104M 0 0 0 0>;
#endif
};
#endif
```

重要的宏定义说明如下：

- **CONFIG_USE_EMMC** 宏用于配置要识别的设备。
 - 1 识别 eMMC 设备
 - 0 识别 SD 卡
- **CONFIG_USE_EMMC_SD_ADAPTIVE** 宏用于配置通过 GPIO 识别安装的存储设备。
 - 1 使能通过 GPIO 识别安装的存储设备
 - 0 关闭通过 GPIO 识别安装的存储设备

若客户设备同时安装了 SD 卡和 eMMC 设备，需要通过 GPIO 识别相应的设备，则需先定义 **CONFIG_USE_EMMC** 宏为 1，然后定义宏 **CONFIG_USE_EMMC_SD_ADAPTIVE** 宏为 1 以使能模块通过 GPIO 识别存储设备。用于识别的引脚客户可以自定义。

3.2. 内核配置

3.2.1. eMMC 驱动配置

内核配置文件中默认已打开 eMMC 配置，用户无需进行额外操作。

3.2.2. ext4 文件系统

1. 在代码目录下，执行 **make kernel_menuconfig**。示例如下：

```
gabriel@ubuntu:~/ql-ol-extsdk-ag35cctcar01a02n2g_ocpu$ make kernel_menuconfig
if [ ! -f /home/gabriel/ql-ol-extsdk-ag35cctcar01a02n2g_ocpu/ql-ol-kernel/.config ]; then cp -f /home/gabriel/ql-ol-extsdk-ag35cctcar01a02n2g_ocpu/ql-ol-kernel/deconfig_asr1806p301 /home/gabriel/ql-ol-ext
sdk-ag35cctcar01a02n2g_ocpu/ql-ol-kernel/.config; fi
make -C /home/gabriel/ql-ol-extsdk-ag35cctcar01a02n2g_ocpu/ql-ol-kernel ARCH=arm CROSS_COMPILE=/opt/ql_crosstools/ql-ag35-1806e-gcc-8.4.0-glibc-v1-toolchain/bin/arm-openwrt-linux- menuconfig
make[1]: Entering directory '/home/gabriel/ql-ol-extsdk-ag35cctcar01a02n2g_ocpu/ql-ol-kernel'
UPD scripts/kconfig/mconf.cfg
HOSTCC scripts/kconfig/mconf.o
HOSTCC scripts/kconfig/xdialog/checklist.o
HOSTCC scripts/kconfig/xdialog/inputbox.o
HOSTCC scripts/kconfig/xdialog/menubox.o
```

2. 选择“File systems”，查看配置信息显示已默认支持 ext4 文件系统，如下所示：

```
.config - Linux/arm 3.10.33 Kernel Configuration
> File systems
File systems
Arrow keys navigate the menu. <Enter> selects submenus --->. Highlighted letters are hotkeys. Pressing <Y>
includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search.
Legend: [*] built-in [ ] excluded <M> module <> module capable

<> Second extended fs support
<> Ext3 journalling file system support
[*] The Extended 4 (ext4) filesystem
[*] Use ext4 for ext2/ext3 file systems (NEW)
[ ] Ext4 POSIX Access Control Lists (NEW)
[ ] Ext4 Security Labels (NEW)
[ ] Ext4 Encryption (NEW)
[ ] EXT4 debugging support (NEW)
[ ] JBD2 (ext4) debugging support (NEW)
<> Reiserfs support
<> JFS filesystem support
<> XFS filesystem support
<> GFS2 file system support
<> OCFS2 file system support
<> Btrfs filesystem support
<> NILFS2 file system support
<> yaffs2 file system support
[*] Enable POSIX file locking API
+{+}

<Select> < Exit > < Help > < Save > < Load >
```

备注

1. 建议 eMMC 用户使用 Ext4 文件系统。
2. 模块当前默认支持 Ext4 文件系统，可以通过查看 `/proc/filesystems` 属性文件确认是否支持 ext4。

4 eMMC 文件系统制作和功能测试验证

4.1. LTE OPEN EVB 硬件测试环境

默认情况下，LTE OPEN EVB 支持 eMMC，需要在 EVB 上切换 eMMC 开关。

4.2. eMMC 设备识别

在命令行输入如下命令查看 eMMC 设备是否成功识别：

```
ls /dev/mmc*
```

示例如下：

```
root@Openwrt:~# ls /dev/mmc*
/dev/mmcblk0 /dev/mmcblk0boot0 /dev/mmcblk0boot1 /dev/mmcblk0rpb
```

如上图所示，`/dev/`目录下成功识别到 eMMC 存储器件 `mmcblk0`。

4.3. eMMC 设备块大小

在命令行输入如下命令查看 eMMC 的块大小：

```
cat /proc/partitions
```

示例如下：

```
root@Openwrt:~# cat /proc/partitions | grep mmc
179      0      3760128 mmcblk0
179      72         512 mmcblk0rpb
179      48      16384 mmcblk0boot1
179      24      16384 mmcblk0boot0
root@Openwrt:~#
root@Openwrt:~#
```

4.4. 分区管理

进入 Linux 系统后，在 shell 终端输入以下命令，对 eMMC 进行分区：

```
# fdisk /dev/mmcblk0
```

示例如下：

```
root@Openwrt:~# fdisk /dev/mmcblk0
welcome to fdisk (util-linux 2.24.1).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Command (m for help):
```

4.4.1. 查看 eMMC 已有分区

输入命令 **p**，查看 eMMC 已有分区信息。示例如下：

```
Command (m for help): p
Disk /dev/mmcblk0: 3.6 GiB, 3850371072 bytes, 7520256 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x00000000

Device           Boot Start      End  Blocks  Id System
/dev/mmcblk0p1    2048    7520255  3759104  83 Linux
```

由上图可知，当前使用的 eMMC 存在一个分区 mmcblk0p1。

4.4.2. 删除分区

进行删除操作前，需保证 eMMC 设备已存在分区。若 eMMC 已有分区，输入命令 **d**，并依次输入要删

除的分区号。若 eMMC 设备只有一个分区，输入 **d** 默认删除仅有的分区。示例如下：

```
Command (m for help): d
Selected partition 1
Partition 1 has been deleted.

Command (m for help): p
Disk /dev/mmcblk0: 3.6 GiB, 3850371072 bytes, 7520256 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x00000000
```

4.4.3. 创建分区

以创建两个分区为例。第一个分区大小是 50 MB，剩余的存储空间分配给第二个分区。

输入命令 **n**，接着输入命令 **p**，建立第一个分区（大小 50 MB），按 ENTER 键执行当前默认配置。

```
Command (m for help): n
Partition type:
  p   primary (0 primary, 0 extended, 4 free)
  e   extended
Select (default p): p
Partition number (1-4, default 1):
First sector (2048-7520255, default 2048):
Last sector, +sectors or +size[K,M,G,T,P] (2048-7520255, default 7520255):

Created a new partition 1 of type 'Linux' and of size 3.6 GiB.
Command (m for help): █
```

输入命令 **n**，接着输入命令 **p**，再输入命令 **2**，建立第二个分区（大小为 eMMC 剩余空间大小），按 ENTER 键执行当前默认配置。输入 **p**，查看已创建的分区信息。

4.4.4. 写入新分区信息

输入命令 **w**，把新建分区信息写入并保存至 eMMC 设备。示例如下：

```
Command (m for help): p
Disk /dev/mmcblk0: 3.6 GiB, 3850371072 bytes, 7520256 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x00000000

Device            Boot Start      End  Blocks  Id System
/dev/mmcblk0p1    2048    7520255  3759104  83 Linux
```

4.4.5. 格式化分区为 ext4 文件系统

此处以分区 1 为例，即 mmcblk0p1 分区，演示使用制作的 *mke2fs* 工具将分区 1 格式化为 ext4 文件系统。

```
# mke2fs -t ext4 /dev/mmcblk0p1
```

```
root@openwrt:/# mke2fs -t ext4 /dev/mmcblk0p1
mke2fs 1.42.4 (12-June-2012)
Discarding device blocks: done
Filesystem label=
OS type: Linux
Block size=4096 (log=2)
Fragment size=4096 (log=2)
Stride=0 blocks, stripe width=0 blocks
235248 inodes, 939776 blocks
46988 blocks (5.00%) reserved for the super user
First data block=0
Maximum filesystem blocks=964689920
29 block groups
32768 blocks per group, 32768 fragments per group
8112 inodes per group
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736

Allocating group tables: done
writing inode tables: done
Creating journal (16384 blocks): done
writing superblocks and filesystem accounting information: done

root@openwrt:/#
```

备注

ext4 文件系统制作工具 *mke2fs* 默认已集成于模块固件中。若未集成，请联系移远通信技术支持。

4.5. 分区读写测试

将 eMMC 分区 1 挂载到 */mnt/mmcblk0p1* 目录下。

```
root@openwrt:/#
root@openwrt:/# mount -t ext4 /dev/mmcblk0p1 /mnt/mmcblk0p1/
[14465.565338] EXT4-fs (mmcblk0p1): mounted filesystem with ordered data mode. opts: (null)
root@openwrt:/#
```

使用 **df -h | grep mmc** 查看 eMMC 分区 1 是否挂载成功，以及分区格式是否正确。

```
root@openwrt:/# df -h | grep mmc
/dev/mmcblk0p1      3.5G      7.2M      3.3G    0% /mnt/mmcblk0p1
root@openwrt:/#
```

执行如下命令，生成大小为 3 GB 的文件并存储至/mnt/mmcblk0p1/，即 eMMC 分区 1：

```
dd if=/dev/zero of=/mnt/mmcblk0p1/test1.img bs=1M count=3072
```

查看新生成的文件大小，如果大小正确，可以判定 eMMC 读写成功。

```
root@Openwrt:/# dd if=/dev/zero of=/mnt/mmcblk0p1/test1.img bs=1M count=3072
3072+0 records in
3072+0 records out
root@Openwrt:/# df -h
Filesystem                Size      Used Available Use% Mounted on
rootfs                    128.8M    112.0K    124.0M   0% /
/dev/root                   7.5M       7.5M        0 100% /rom
tmpfs                      47.9M     80.0K    47.8M   0% /tmp
ubi0_0                     128.8M    112.0K    124.0M   0% /overlay
ubi1:oem_data               3.1M     20.0K     3.1M   1% /NVM/oem_data
overlayfs:/overlay/root
tmpfs                      128.8M    112.0K    124.0M   0% /
tmpfs                      512.0K        0    512.0K   0% /dev
/dev/mmcblk0p1             3.5G      3.0G    270.5M  92% /mnt/mmcblk0p1
root@Openwrt:/#
```

5 eMMC 常用调试方法

5.1. eMMC 设备读写测试

eMMC 设备被识别后，会在/dev/目录下生成名为 mmcblk0 的设备。可通过执行 **echo**、**hexdump** 等命令对 eMMC 设备进行简单的读写测试，方法如下。

5.1.1. 写入数据

执行如下命令向识别到的 eMMC 设备 (/dev/mmcblk0) 写入数据：

```
echo HelloWorld! > /dev/mmcblk0
```

示例如下图：

```
root@Openwrt:/#
root@Openwrt:/#
root@Openwrt:/# ls /dev/mmc*
/dev/mmcblk0      /dev/mmcblk0boot0 /dev/mmcblk0boot1 /dev/mmcblk0rpb
root@Openwrt:/#
root@Openwrt:/#
root@Openwrt:/# echo HelloWorld! > /dev/mmcblk0
root@Openwrt:/#
```

5.1.2. 读取数据

执行如下命令从识别到的 eMMC 设备 (/dev/mmcblk0) 读取数据：

```
hexdump -C -n 15 /dev/mmcblk0
```

示例如下图：

```
root@Openwrt:/#
root@Openwrt:/# hexdump -C -n 15 /dev/mmcblk0
00000000 48 65 6c 6c 6f 57 6f 72 6c 64 21 0a 00 00 00 |HelloWorld!....|
0000000f
root@Openwrt:/#
```

5.2. 查看当前系统支持的文件系统

执行如下命令查看当前 Linux 系统支持的文件系统类型：

```
cat /proc/filesystems
```

如下图可知，当前系统支持 vfat、ext2、ext3、ext4 等文件系统。



```
root@Openwrt:/#
root@Openwrt:/# cat /proc/filesystems
nodev    sysfs
nodev    rootfs
nodev    bdev
nodev    proc
nodev    tmpfs
nodev    debugfs
nodev    sockfs
nodev    pipefs
nodev    anon_inodefs
nodev    configfs
nodev    devpts
nodev    ext3
nodev    ext2
nodev    ext4
nodev    squashfs
nodev    ramfs
nodev    vfat
nodev    fuseblk
nodev    fuse
nodev    fusectl
nodev    overlayfs
nodev    mqueue
nodev    mtd_inodefs
nodev    ubifs
root@Openwrt:/#
```

5.3. 获取 eMMC 基本信息

识别 eMMC 时，eMMC 驱动代码已将一些常用信息通过文件的形式创建在/sys/目录下。用户可以通过 cat 等命令查看设备信息。如下为常用命令：

```
ls /sys/block/mmcblk0 -l
cat /sys/block/mmcblk0/size
cat /proc/self/mounts
cat /sys/block/mmcblk0/device/cid, csd, date, fwrev, hwrev, manfid, name, oemid, serial, type,
uevent
```

5.4. Linux MMC 驱动子系统简述

5.4.1. Linux MMC 子系统介绍

MMC、SD、SDIO 三种技术均起源于 MMC 技术，有很多共性，因此 Linux Kernel 统一使用 MMC framework 管理所有和这三种技术有关的设备。

Linux 的 MMC 子系统在内核 `drivers/mmc/` 目录中实现，该目录下有 3 个子目录：`card`、`core` 和 `host`，分别用于存放 MMC 卡的块设备、核心层和主控制器的相关代码，并反映该子系统的层次架构。

MMC 子系统从上到下可分为 3 个层次：Host 层、Core 层和 Card 层。这 3 个层次的作用定义如下：

- **Host 层：**
也叫主控制器驱动层。主控制器通常是 SoC 上的一个外设，通过它，SoC 能够按照 MMC 协议规定的方式与卡进行数据传输。在 `drivers/mmc/host/` 子目录下有很多主控制器驱动，其中包括模块的 SDIO 驱动 `sdhci-msm.c`。
- **Core 层：**
即 MMC 子系统核心层，它的实现代码位于 `drivers/mmc/core/` 子目录中。核心层（主控制器驱动程序）除为 Host 层驱动提供诸如 `mmc_alloc_host()/mmc_add_host()/mmc_remove_host()` 等分配、添加、删除 MMC 主控制器对象的接口外，还分别就 MMC 和 SDIO 两种规范实现了相应的协议代码。
- **Card 层：**
因为 MMC 卡属于块设备，所以需要为这些设备提供块设备驱动。在 `drivers/mmc/core/` 中，`block.c` 主要实现了 MMC 卡的通用块设备驱动，在 `queue.c` 中则实现了该块设备的请求队列及处理。MMC 卡块设备的主设备号是 179。

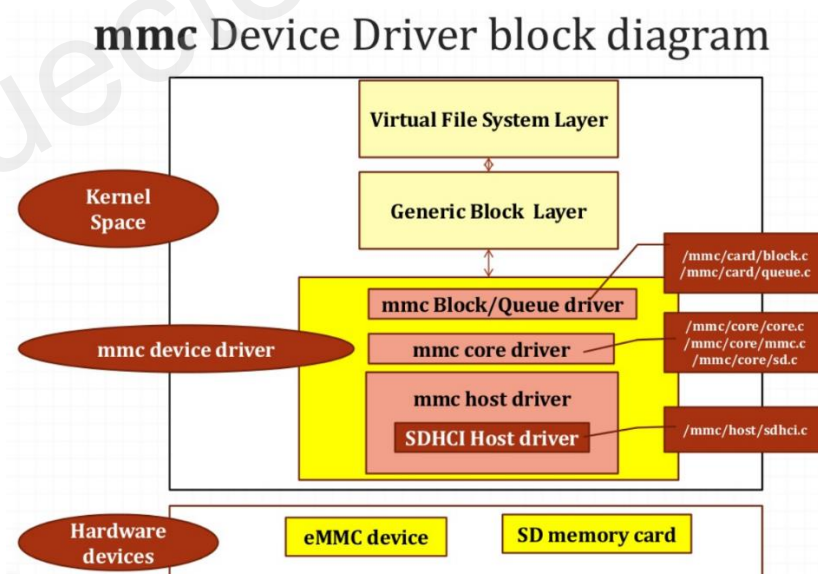


图 1: Linux MMC 子系统架构图

5.4.2. Linux MMC 子系统驱动初始化流程

通常，大多数问题出现在驱动初始化过程中，整理 MMC 驱动框架的初始化过程如下：

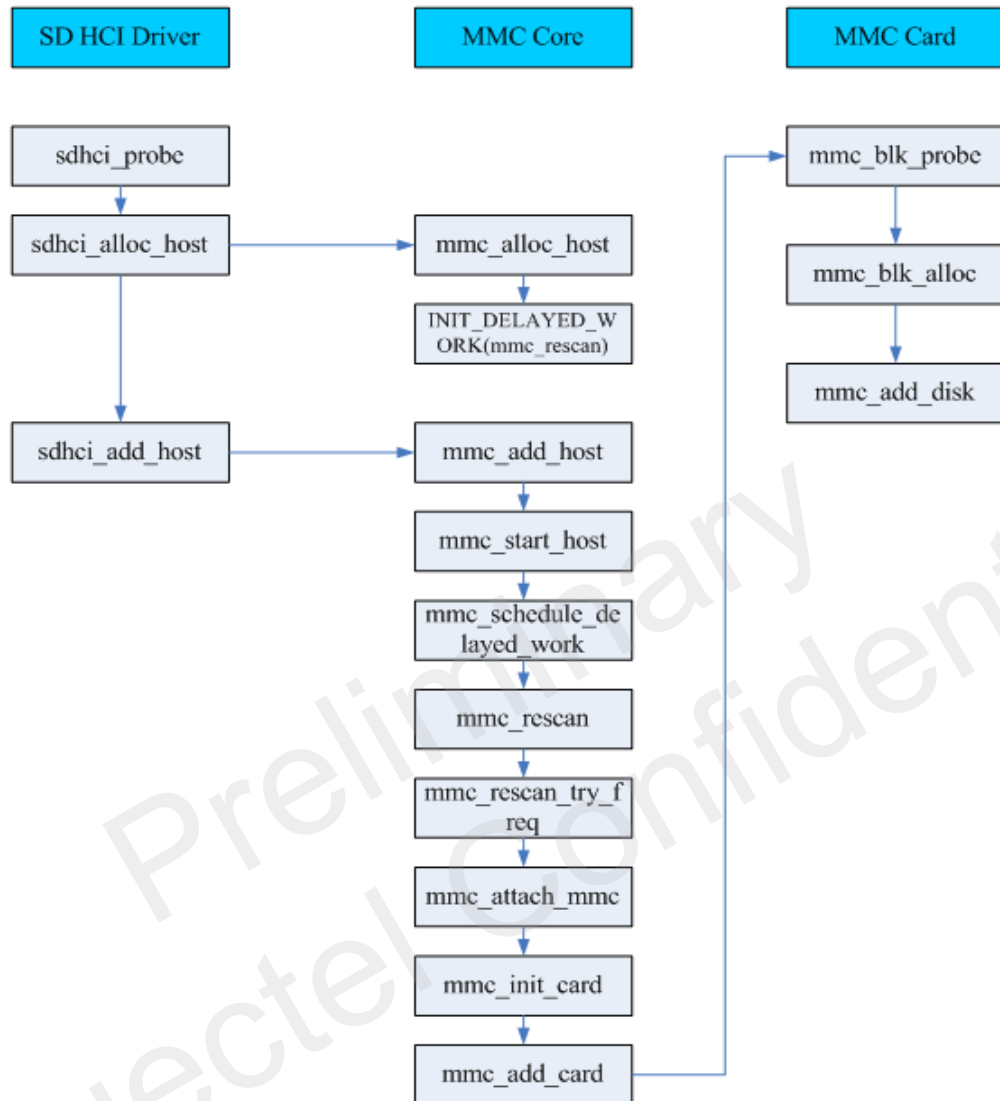


图 2: Linux MMC 驱动初始化流程图

5.5. 打开调试信息

5.5.1. 打开 MMC 子系统调试信息

```

46 #include "mmc_ops.h"
47 #include "sd_ops.h"
48 #include "sdio_ops.h"
49
50 #undef pr_debug
51 #define pr_debug pr_err
52
53 /* The max erase timeout, used when host->max_busy_timeout isn't specified */
54 #define MMC_ERASE_TIMEOUT_MS (60 * 1000) /* 60 s */
55 #define SD_DISCARD_TIMEOUT_MS (250)

```

如上图所示，在 `drivers/mmc/core/core.c` 文件中添加如下宏定义：

```

#undef pr_debug
#define pr_debug pr_err

```

5.5.2. 打印 MMC 命令行 CMD log 信息

```

ql-ol-kernel > drivers > mmc > M Makefile
1  # SPDX-License-Identifier: GPL-2.0-only
2  #
3  # Makefile for the kernel mmc device drivers.
4  #
5
6  obj-$(CONFIG_MMC) += core/
7  obj-$(subst m,y,$(CONFIG_MMC)) += host/
8
9  subdir-ccflags-y = -DDEBUG
10

```

如上图所示，`/ql-ol-kernel/drivers/mmc/Makefile` 增加上面这行

```
subdir-ccflags-y = -DDEBUG
```

6 Mdev 自动挂载

6.1. mdev 简介

mdev 是 *busybox* 提供的一个用在嵌入式系统中的工具，相当于简化版的 *udev*，用于在系统启动和热插拔或动态加载驱动程序时，自动创建设备节点。文件系统中 */dev/* 目录下的设备节点都是由 *mdev* 创建的。

6.2. eMMC 自动挂载机制

eMMC 采用 *mdev* 的机制进行自动挂载。*mdev* 配置文件路径为 *ql-ol-rootfs/etc/mdev.conf*，相关 eMMC 配置如下所示；当识别到了 eMMC 设备且存在 eMMC 分区，就会运行 */etc/mdev/automountsdcard.sh* 脚本。

```
tun[0-9]* 0:0 0660
mmcblk[0-9]* 0:6 660
mmcblk[0-9]*p[0-9]* 0:6 660 */etc/mdev/automountsdcard.sh ${MDEV}
[hs]d[a-z][0-9]* 0:0 660 */etc/mdev/automountsdcard.sh ${MDEV}
```

使用 *df -Th* 命令查看分区 */dev/mmcblk0p1* 和 */dev/mmcblk0p2* 挂载的目录分别为 */mnt/mmcblk0p1/* 和 */mnt/mmcblk0p2/*。

```
root@openwrt:~# df -Th
Filesystem      Type      Size      Used Available Use% Mounted on
/dev/dm-0        squashfs  13.3M      13.3M      0 100% /
tmpfs            tmpfs     45.5M      116.0K    45.4M   0% /tmp
ubi0:data        ubifs     90.7M      28.0K     86.0M   0% /data
ubi0:data        ubifs     90.7M      28.0K     86.0M   0% /mnt
ubi0:data        ubifs     90.7M      28.0K     86.0M   0% /log
ubi0:etc         ubifs     2.9M       1.9M     856.0K  69% /overlay/etc
ubi0:nvm         ubifs     2.0M       56.0K     1.8M    3% /overlay/nvm
overlayfs:/overlay/etc overlay    2.9M       1.9M     856.0K  69% /etc
overlayfs:/overlay/nvm overlay    2.0M       56.0K     1.8M    3% /NVM
/dev/ubiblock1_0 squashfs  128.0K     128.0K      0 100% /NVM/oem_data
tmpfs            tmpfs     512.0K      0    512.0K   0% /dev
/dev/mmcblk0p2   ext4      3.4G       32.0K     3.2G    0% /mnt/mmcblk0p2
/dev/mmcblk0p1   ext4      43.6M      16.0K     40.1M    0% /mnt/mmcblk0p1
root@openwrt:~#
root@openwrt:~#
```

若不使用 *mdev* 的机制挂载 eMMC，可删除 *mdev* 自动挂载 eMMC 机制，注释掉 *ql-ol-rootfs/etc/mdev.conf* 配置文件的 eMMC 自动挂载配置，如下图所示；如需使用 *mdev* 机制自动挂载 eMMC，则可直接修改 */etc/mdev/automountsdcard.sh* 自动挂载脚本。

```

can[0-9]* 0:0 660
mmcblk[0-9]* 0:6 660
#mmcblk[0-9]*p[0-9]* 0:6 660 */etc/mdev/automountsdcard.sh ${MDEV}
[hs]d[a-z][0-9]* 0:0 660 */etc/mdev/automountsdcard.sh ${MDEV}

```

7 附录 参考文档及术语缩写

表 2: 参考文档

文档名称
[1] Quectel_AG35-CET_QuecOpen(SDK)_快速开发指导
[2] Quectel_AG35-CET_QuecOpen_硬件设计手册

表 3: 术语缩写

缩写	英文全称	中文全称
eMMC	Embedded Multi Media Card	嵌入式多媒体控制器
Ext4	Fourth extended filesystem	第四代扩展文件系统
FAT	File Allocation Table	文件配置表
IoV	Internet of Vehicles	车联网
SD Card	Secure Digital Memory Card	安全数码卡
SOC	System On Chip	片上系统